

How Does Wilcoxon Rank Sum Test Perform in Single-Cell RNA-seq Differential Expression Analysis?

Yifan Zhao¹

Abstract

Differentially expressed genes (DEGs) are crucial to identifying cell clusters in single-cell RNA-seq data analysis. Here we analyze how well Wilcoxon tests perform in correctly distinguishing DEGs from non-DEGs using simulated single-cell expressions.

Keywords: simulation, DEG, single-cell RNA-seq, Wilcoxon test

Introduction

Compared to traditional bulk RNA-seq experiments, single-cell RNA-sequencing (scRNA-seq) experiments allow us to examine the gene expression level at a finer resolution. In order to discover cell functions, marker genes are usually identified at the level of clusters. Such markers are helpful for cell annotation, cellular communication analysis, trajectory analysis, and so on. However, finding differentially expressed genes (DEGs) is controversial in that scRNA-seq experiments often contain a large proportion of 0 counts for some genes due to the nature of cell type as well as drop-out events. Besides, the underlying distribution for each gene in each cell may not follow normal distributions. In order to find DEGs, many tools have been developed, including tools using parametric methods, such as SCDE, edgeR, DESeq2, DESingle, and so on, as well as tools using non-parametric methods, such as EMDomics, SigEMD, and SINCERA¹.

Motivating Problem

According to Wang *et al.*¹, because of the complexity of scRNA-seq data mentioned in the Introduction, methods accounting for multi-modality (non-parametric methods) perform better than those relying on specific models to handle zero counts in their study. As one of the most popular non-parametric methods, Wilcoxon Rank Sum (WRS) Tests have been applied to many scenarios. Here, we would like to assess how WRS tests perform in detecting DEGs in simulated data.

¹Department of Biostatistics and Informatics, Colorado School of Public Health, University of Colorado Anschutz Medical Campus, Aurora, CO

Simulation Set-up

In order to simulate the scRNA-seq data, we need some assumptions. Here we adopt the same models used by Korshauer *et al.*². Below is a detailed description of **models** and **scenarios** we assume. A visualization summarizing our simulation models can be found in Figure 1.

- **Basic gene expression models:** In a group of cells, it is assumed that there exist two types of basic models. One has a unique mode from a negative binomial distribution (**uni-modal**, 1A), while the other has two modes from two negative binomial distributions with different parameters (**bi-modal**, Figure 1B). The bi-modal distribution represents a shift in the gene expression level from one state to another. These two types of theoretical distributions are the building blocks of the following scenarios.
- **DEG scenarios:**
 - **DE:** Traditional Differential Expression, where gene expression in each group has a uni-modal distribution with different parameters.
 - **DP:** Differential Proportion, where gene expression in each group has bi-modal distribution, with different proportions of cells in each mode. This scenario mimics the case where one group has almost finished the shift in gene expression, while the other has just started.
 - **DM:** Differential Modality, where one group has a uni-modal distribution, and the other has a bi-modal distribution. This scenario represents the case where one group has reached a stable expression level, while the other is experiencing a shift in expression level.
 - **DB:** DP and DM Both, where one group has a uni-modal distribution, while the other has a bi-modal distribution. This scenario differs from DM in that the means of the two groups in DB are very close. This case would be hard to detect for some methods focusing on comparing the means.
- **non-DEG scenarios:**
 - **EE:** uni-modal distribution with the same parameters.
 - **EP:** bi-modal distribution with the same parameters.

Furthermore, in order to mimic the drop-out events, a random percentage (between 25% and 50%) of zeros will replace the simulated counts.

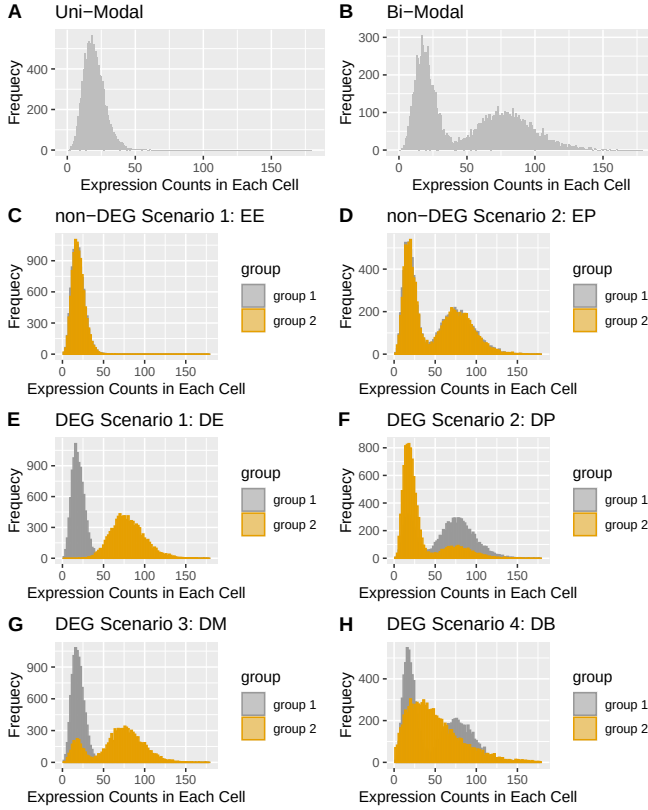


Figure 1: Histograms for all simulated models

Results

In this study, we simulate 75 cells in two groups, each with 10,000 genes. The number 75 is chosen because this represents a medium size cluster in scRNA-seq analysis. Among the 10,000 genes, 8000 are non-DEGs (4000 for each non-DEG model), and 2000 are DEGs (500 for each DEG model). For each gene, Wilcoxon Rank Sum Test has been performed, where the null hypothesis is that the mean rank sums of the two groups are the same, while the alternative hypothesis is that the mean rank sums are different. For each test, the corresponding p-value is recorded. Because we have a relatively large number of tests to perform, adjusted p-values are calculated using the “FDR” correction method. Then, based on a threshold of adjusted p-value = 0.05, we determine if we reject or fail to reject the null hypothesis. A summary table can be found in Table 1.

Further, we calculate the True Positive Rate (TPR) for each scenario in the DEG models, and the True Negative Rate (TNR) for each scenario in the non-DEG models. As shown in Figure 2, we can see that WRS tests perform well in the DE model (TPR = 80.4%) and OK in the DM model (TPR = 50.6%, similar to random guessing). However, it performs poorly in DP and DB models (TPR < 15%). On the other hand, it identifies non-DEG cases with high accuracy.

Table 1: Summary for Simulation Design and Test Results

Model	Scenario	Count	DEG.adj	NonDEG.adj
DEG	DE	500	402	98
	DP	500	63	437
	DM	500	253	247
	DB	500	74	426
non-DEG	EE	4000	57	3943
	EP	4000	14	3986

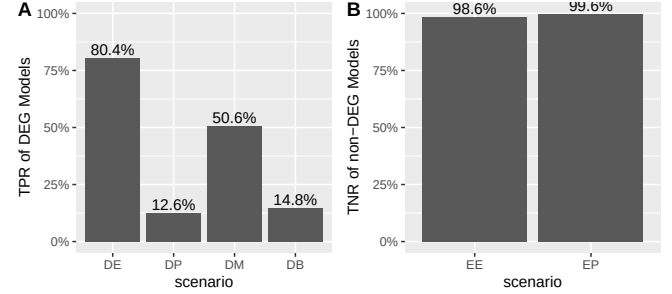


Figure 2: True Detection Rates for Each Scenario

Conclusion

The result matches our intuition that WRS is able to detect DEGs in differentially expressed uni-modal cases (DE Model), while it is not good at identifying DEGs in complex cases where bi-modal cases appear (DP, DM, and DB). Our conclusion is similar to the results of the non-parametric tool SINCERA tested by Wang *et al.*¹. Therefore, alternative tools with better performance are recommended to detect DEGs instead of WRS.

However, our conclusion is far from comprehensive because of several limitations of this simulation study. First, this study is based on strong assumptions of the underlying models of gene expression in a cell population, such as the characteristics of drop-out and multi-modality. On top of that, many parameters can be further refined. For example, we only considered 75 cells in each cluster, which is not always the case. Perhaps the performance will improve when a larger number of cells are considered. The parameters for the basic gene models (mean and variance) were arbitrarily chosen without a good reason. More careful design, such as using parameters from real data, could have been applied. Furthermore, in real-world analysis, several scRNA-seq datasets are often combined using integration algorithms. In this case, simply using the original molecule counts may not represent such analysis settings well. Despite the limitations of our study, we speculate that WRS tests will not perform exceptionally well or robustly, considering the nature of WRS tests.

References

1. Wang T, Li B, Nelson CE, Nabavi S. Comparative analysis of differential gene expression analysis tools for single-cell RNA sequencing data. BMC Bioinformatics 2019;20:1–6.
2. Korthauer KD, Chu L-F, Newton MA, Li Y, Thomson J, Stewart R, et al. A statistical approach for identifying differential distributions in single-cell RNA-seq experiments. Genome Biology 2016;17:1–5.

Acknowledgement

First of all, thank Professor Alex Kaizer for giving me the chance to carry out this simulation study with gracious extension on the deadline of this project. We would also like to thank Nannan Wang for providing helpful suggestions to this study. In addition, we thank Zixuan Wei, Xuedan Li, and Wendan Zhang for moral support. *Soli Deo gloria.*

Supplementary Material

Table S1 is a full table of summary for our test results, which includes the counts of cells identified as DEGs and non-DEGs before applying the multiple comparison method.

Table S1: Full Summary Table for Simulation Design and Test Results

Model	Scenario	Count	Test Results Based on p.adj			
			DEG	DEG.adj	NonDEG	NonDEG.adj
DEG	DE	500	474	402	26	98
	DP	500	145	63	355	437
	DM	500	362	253	138	247
	DB	500	189	74	311	426
non-DEG	EE	4000	465	57	3535	3943
	EP	4000	392	14	3608	3986

Code Appendix

Code for Helper Functions

```
# Helper Functions -----
## Simulation Set Up -----
# nbiom_param: Generate negative binomial parameters, size and prob
# Input:
#   mu: mean of negative binomial model
#   variance: variance of the desired model
# Output:
#   size: number of successes
#   prob: probability of success at each trial
nbio_param <- function(mu, variance) {
  prob <- mu / variance
  size <- mu * prob / (1-prob)
  if(size <= 0) {
    stop("Size is negative.")
  }
  if(prob >= 1 | prob <= 0) {
```

```

        stop("Prob is not between 0 and 1.")
    }
    return(list(size = size, prob = prob))
}

# sim_unimodal: Simulate data from a uni-modal negative binomial distribution
# Input:
#   n: number of values to generate
#   size: number of successes in negative binomial model
#   prob: probability for each trial to be a success
# Output:
#   sim: a vector of simulated data
sim_unimodal <- function(n, size, prob) {
    sim <- rnbinom(n = n, size = size, prob = prob)
    # hist(sim)
    # print(mean(sim))
    # print(var(sim))
    return(sim)
}

# sim_bimodal: Simulate data from a bi-modal negative binomial distribution
# Input:
#   n: number of values to generate
#   prop1: proportion of data points from modal 1, default 0.5
#   size1/2: number of successes in negative binomial model in modal 1/2
#   prob1/2: probability for each trial to be a success in modal 1/2
# Output:
#   sim: a vector of simulated data
sim_bimodal <- function(n, prop1 = 0.5, size1, prob1, size2, prob2) {
    # floor has to be used; otherwise, it may result 1 fewer result
    # eg. n = 75, prop = 0.5, n1 = 37.5, n2 = 37.5
    # => 37 for each, 74 in total
    n1 <- floor(n * prop1)
    n2 <- n - n1
    sim1 <- c(rnbinom(n = n1, size = size1, prob = prob1), rep(0, n2))
    sim2 <- c(rep(0, n1), rnbinom(n = n2, size = size2, prob = prob2))
    sim <- sim1 + sim2
    # hist(sim)
    return(sim)
}

# pop_zeros: Populate a certain percent of zeros in given simulated data
# Input:
#   sim_data: a vector of simulated data
#   percent: percent of zeros to populate; if not specified, it will generate
#             a random number between 0.25 and 0.5 from a uniform distribution
# Output:
#   sim_data: a vector of simulated data with certain percent of zeros
pop_zeros <- function(sim_data, percent = runif(1, min = 0.25, max = 0.5)) {
    n <- length(sim_data)
    index <- sample(n, size = n*percent)
    # Note: the function sample is able to deal with size = non-integers
    # For example: sample(10, 2.4) will give you 2 numbers

```

```

    # Replace the randomly selected indices with 0
    sim_data[index] <- 0
    return(sim_data)
}

# sim_uni_zeros: Simulate uni-modal data with a certain percent of zeros
# Input:
#   n: number of values to generate
#   size: number of successes in negative binomial model
#   prob: probability for each trial to be a success
#   zero_pct: percent of zeros to populate; if not specified, it will generate
#             a random number between 0.25 and 0.5 from a uniform distribution
# Output:
#   sim: a vector of simulated data with certain percent of zeros
sim_uni_zeros <- function(n, size, prob, zero_pct = NULL) {
  sim_data <- sim_unimodal(n, size = size, prob = prob)

  # Note: tried to use ifelse() function instead, but it didn't work.
  if (is.null(zero_pct)) {
    sim_data <- pop_zeros(sim_data)
  } else {
    sim_data <- pop_zeros(sim_data, percent = zero_pct)
  }

  return(sim_data)
}

# sim_bi_zeros: Simulate bi-modal data with a certain percent of zeros
# Input:
#   n: number of values to generate
#   prop1: proportion of data points from modal 1, default 0.5
#   size1/2: number of successes in negative binomial model in modal 1/2
#   prob1/2: probability for each trial to be a success in modal 1/2
#   zero_pct: percent of zeros to populate; if not specified, it will generate
#             a random number between 0.25 and 0.5 from a uniform distribution
# Output:
#   sim: a vector of simulated data with certain percent of zeros
sim_bi_zeros <- function(n, prop1 = 0.5, size1, prob1, size2, prob2,
  zero_pct = NULL) {
  sim_data <- sim_bimodal(n = n, prop1 = prop1,
    size1 = size1, prob1 = prob1,
    size2 = size2, prob2 = prob2)

  if (is.null(zero_pct)) {
    sim_data <- pop_zeros(sim_data)
  } else {
    sim_data <- pop_zeros(sim_data, percent = zero_pct)
  }

  return(sim_data)
}

## Wilcoxon Test -----
suppressPackageStartupMessages(library(coin))

```

```

# wilcoxon_wrap: Given a list of two groups, perform wilcoxon rank sum test,
# returns a list: p (p-value) and DEG (True or False)
wilcoxon_wrap <- function(data_list) {
  grp1 <- data_list$grp1
  grp2 <- data_list$grp2
  combined <- data.frame(exp = c(grp1, grp2),
                          grp = factor(
                            c(rep("grp1", length(grp1)),
                              rep("grp2", length(grp2)))))

  test_result <- wilcox_test(exp ~ grp, combined, distribution = 'exact')
  p <- pvalue(test_result)

  isDEG <- pvalue(test_result) < 0.05

  return(list(p = p, DEG = isDEG))
}

## Models -----
# get_summary: Get the summary statistics for each element in the input list
get_summary <- function(data_list) {
  for (i in names(data_list)) {
    data_i <- data_list[[i]]
    # Print summary statistics
    cat(paste("Summary statistics for element", i, "\n"))
    print(summary(data_i))

    # Print length
    cat("Length: \n")
    print(length(data_i))

    # Print percent of zeros
    cat("Percentage of zeros: \n")
    print(mean(data_i == 0))

    # Histogram
    hist(data_i)
  }
}

# sim_EE: [non-DEG (EE)] Simulate two sets of data using the same distribution
sim_EE <- function(n, size, prob, zero_pct = NULL) {
  result <- list()
  for (i in c("grp1", "grp2")) {
    result[[i]] <- sim_uni_zeros(n = n, size = size,
                                prob = prob, zero_pct = zero_pct)
  }

  return(result)
}

# sim_EP: [non-DEG (EP)] Simulate two sets of data with two modes,
# equivalent of cells belonging to each component.
sim_EP <- function(n, size1, prob1, size2, prob2, zero_pct = NULL) {
  result <- list()

```

```

    for (i in c("grp1", "grp2")) {
      result[[i]] <- sim_bi_zeros(n = n, size1 = size1, prob1 = prob1,
                                size2 = size2, prob2 = prob2,
                                zero_pct = zero_pct)
    }
    return(result)
  }
}

```

```

# sim_DE: [DEG (DE)] Simulate two sets of data with one mode in each
sim_DE <- function(n, size1, prob1, size2, prob2, zero_pct = NULL) {
  grp1 <- sim_uni_zeros(n = n, size = size1,
                        prob = prob1, zero_pct = zero_pct)
  grp2 <- sim_uni_zeros(n = n, size = size2,
                        prob = prob2, zero_pct = zero_pct)
  return(list(grp1 = grp1, grp2 = grp2))
}

```

```

# sim_DP: [DEG (DP)] Simulate two sets of data with two modes in each,
# differential proportion of each mode
# Because the two modes in each group come from the same distribution,
# the size1/2 and prob1/2 are shared across groups.
sim_DP <- function(n, prop1.1, prop2.1, size1, prob1, size2, prob2,
                  zero_pct = NULL) {
  results <- list()
  props <- c(prop1.1, prop2.1)
  for (i in 1:2) {
    id <- paste0("grp", i)
    results[[id]] <- sim_bi_zeros(n = n, prop1 = props[i],
                                  size1 = size1, prob1 = prob1,
                                  size2 = size2, prob2 = prob2,
                                  zero_pct = zero_pct)
  }
  return(results)
}

```

```

# sim_DM: [DEG (DM)] Simulate two sets of data with differential number of modes
# Because they share a same mode (size_cmn, prob_cmn),
# we need an extra set of parameters
sim_DM <- function(n, size_cmn, prob_cmn, zero_pct_uni = NULL,
                  prop_ext, size_ext, prob_ext, zero_pct_bi = NULL) {
  # Simulate uni-modal data
  grp1 <- sim_uni_zeros(n = n, size = size_cmn, prob = prob_cmn,
                        zero_pct = zero_pct_uni)
  # Simulate bi-modal data
  grp2 <- sim_bi_zeros(n = n, prop1 = prop_ext,
                       size1 = size_cmn, prob1 = prob_cmn,
                       size2 = size_ext, prob2 = prob_ext,
                       zero_pct = zero_pct_bi)
  return(list(grp1 = grp1, grp2 = grp2))
}

```

```

# sim_DB: [DEG (DB)] Simulate two sets of data both DM and differential means

```

```

# of expressions
# Because given a bi-modal distribution, we can calculate the mean and variance,
# and thus we can simulate a uni-modal distribution with similar mean and var,
# only a set of parameters for bi-modal distribution should be provided
sim_DB <- function(n, size1, prob1, size2, prob2, zero_pct = NULL) {
  # Simulate bi-modal data
  # Note: for this model, we do not allow the change of proportion for
  # mode 1 (prop), which will use the default 0.5.
  grp1 <- sim_bi_zeros(n = n, size1 = size1, prob1 = prob1,
                      size2 = size2, prob2 = prob2, zero_pct = zero_pct)

  # Simulate uni-modal data based on the same mean and variance
  grp2_params <- nbioM_param(mu = mean(grp1), variance = var(grp1))
  grp2 <- sim_uni_zeros(n = n, size = grp2_params$size,
                      prob = grp2_params$prob, zero_pct = zero_pct)

  return(list(grp1 = grp1, grp2 = grp2))
}

```

Test Cases for Helper Functions

```

# Source helper functions
source("HelperFunctions.R")

# Set seed for reproducibility of the test cases
set.seed(6611)

## nbioM_param -----
param1 <- nbioM_param(mu = 20, variance = 60)
sim1 <- rnbinom(n = 3000, size = param1$size, param1$prob)
# Note: here we set a big n so that we can see easily whether the data follow a
# negative binomial distribution
hist(sim1)

param2 <- nbioM_param(mu = 80, variance = 400)
sim2 <- rnbinom(n = 3000, size = param2$size, param2$prob)
hist(sim2)

hist(sim1 + sim2)
# Theorem: If the probs in the two models are the same, the sum of two negative
# binomials will become another negative binomial.

## sim_unimodal -----
sim_unimodal(3000, size = 10, prob = 0.3)

## sim_bimodal -----
sim_bimodal(3000, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)

## pop_zeros -----
summary(sim1)
# Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
# 2.00  14.00   19.00   19.98  25.00   58.00

length(sim1)

```

```

# 3000

sum(sim1 == 0)
# 0

# Case 1: Specify the percent as 0.3
pop_sim1 <- pop_zeros(sim1, percent = 0.3)
summary(pop_sim1)
# Min. 1st Qu. Median Mean 3rd Qu. Max.
# 0.00 0.00 15.00 13.94 22.00 58.00

length(pop_sim1)
# 3000

sum(pop_sim1 == 0)
# 900

# Case 2: Use default as the percent
pop_sim2 <- pop_zeros(sim1)
summary(pop_sim2)
# Min. 1st Qu. Median Mean 3rd Qu. Max.
# 0.00 0.00 16.00 14.95 23.00 58.00

length(pop_sim2)
# 3000

sum(pop_sim2 == 0)
# 758

# sim_uni_zeros -----
# Case 1: unspecified zero_pct
sim_uni_zeros1 <- sim_uni_zeros(n = 100, size = 10, prob = 0.3)

summary(sim_uni_zeros1)
# Min. 1st Qu. Median Mean 3rd Qu. Max.
# 0.00 0.00 18.50 16.11 25.25 39.00

length(sim_uni_zeros1)
# 100

mean(sim_uni_zeros1 == 0)
# 0.31

# Case 2: specified zero_pct
sim_uni_zeros2 <- sim_uni_zeros(n = 100, size = 10, prob = 0.3, zero_pct = 0.6)

summary(sim_uni_zeros2)
# Min. 1st Qu. Median Mean 3rd Qu. Max.
# 0.00 0.00 0.00 9.42 21.50 37.00

length(sim_uni_zeros2)
# 100

mean(sim_uni_zeros2 == 0)
# 0.6

```

```

# sim_bi_zeros -----
# Case 1: unspecified zero_pct
sim_bi_zeros1 <- sim_bi_zeros(n = 100, size1 = 10, prob1 = 0.3,
                             size2 = 50, prob2 = 0.1)

summary(sim_bi_zeros1)
# Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
# 0.00   0.00   18.50   16.11   25.25   39.00

length(sim_bi_zeros1)
# 100

mean(sim_bi_zeros1 == 0)
# 0.31

# Case 2: specified zero_pct
sim_bi_zeros2 <- sim_bi_zeros(n = 100, size1 = 10, prob1 = 0.3,
                              size2 = 50, prob2 = 0.1, zero_pct = 0.6)

summary(sim_bi_zeros2)
# Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
# 0.00   0.00   18.50   16.11   25.25   39.00

length(sim_bi_zeros2)
# 100

mean(sim_bi_zeros2 == 0)
# 0.31

# sim_EE -----
sim_EE1 <- sim_EE(50, size = 10, prob = 0.3)
get_summary(sim_EE1)

# sim_EP -----
sim_EP1 <- sim_EP(n = 50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
get_summary(sim_EP1)

sim_EP2 <- sim_EP(n = 100, size1 = 10, prob1 = 0.05, size2 = 90, prob2 = 0.05)
get_summary(sim_EP2)

# sim_DE -----
sim_DE1 <- sim_DE(n = 50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
get_summary(sim_DE1)

# sim_DP -----
sim_DP1 <- sim_DP(50, prop1.1 = 0.3, prop2.1 = 0.8,
                  size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
get_summary(sim_DP1)

# sim_DM -----
sim_DM1 <- sim_DM(n = 50, size_cmn = 10, prob_cmn = 0.3,
                  prop_ext = 0.2, size_ext = 50, prob_ext = 0.1)
get_summary(sim_DM1)

# sim_DB -----

```

```
sim_DB1 <- sim_DB(50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
get_summary(sim_DB1)
```

Code for Simulation

```
source("HelperFunctions.R")

# Set up:
# Here we simulate 75 cells, with 10,000 genes in total.
# 8,000 genes are non-DEGs: 4,000 EE, 4,000 EP
# 2,000 genes are DEGs: 500 DE, 500 DP, 500 DM, 500 DB

## Initialize matrices to store results -----
sim_data_grp1 <- matrix(NA, nrow = 75, ncol = 10000)
sim_data_grp2 <- matrix(NA, nrow = 75, ncol = 10000)
wilcoxon_p <- rep(NA, 10000)
wilcoxon_jdg <- rep(NA, 10000)

## Simulation Parameters -----
# size: 10, 15, 20, 25, 30, 40, ..., 140, 150, ..., 200, 250, 300, ..., 450, 500
sizes <- c(seq(10, 25, 5), seq(30, 140, 10), seq(150, 500, 50))
length(sizes) # 24

# probs: 0.05, 0.1, 0.2, 0.3, ..., 0.9
probs <- c(0.05, seq(0.1, 0.9, 0.1))
length(probs) # 10

combo <- expand.grid(sizes, probs)
dim(combo)
colnames(combo) <- c("size", "prob")

# populate the rest of the 160 by resampling
set.seed(6611)
resample_index <- sample(1:240, size = 260, replace = T)
resample_params <- combo[resample_index,]

combo <- rbind(combo, resample_params)

dim(combo)
# 500 2
rownames(combo) <- 0:499

# combo$mean <- combo$size * (1-combo$prob) / combo$prob
# combo$variance <- combo$mean / combo$prob

## EE -----
# [non-DEG (EE)] Generate 4000 pairs of data sets for EE,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 1:4000) {
  row_idx <- as.character(i %% 500)
  sim_temp <- sim_EE(75, size = combo[row_idx, "size"],
                    prob = combo[row_idx, "prob"])
}
```

```

sim_data_grp1[, i] <- sim_temp$grp1
sim_data_grp2[, i] <- sim_temp$grp2

wilcox_tmp <- wilcoxon_wrap(sim_temp)

wilcoxon_p[i] <- wilcox_tmp$p
wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

## EP -----
# [non-DEG (EP)] Generate 4000 pairs of data sets for EP,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 4001:8000) {
  row_idx1 <- as.character((i-1) %% 500)
  row_idx2 <- as.character((i+9) %% 500) # 9 is arbitrary
  # e.g. sim_EP(n = 50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
  sim_temp <- sim_EP(n = 75,
                    size1 = combo[row_idx1, "size"],
                    prob1 = combo[row_idx1, "prob"],
                    size2 = combo[row_idx2, "size"],
                    prob2 = combo[row_idx2, "prob"])
  sim_data_grp1[, i] <- sim_temp$grp1
  sim_data_grp2[, i] <- sim_temp$grp2

  wilcox_tmp <- wilcoxon_wrap(sim_temp)

  wilcoxon_p[i] <- wilcox_tmp$p
  wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

## DE -----
# [DEG (DE)] Generate 500 paris of data sets for DE,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 8001:8500) {
  row_idx1 <- as.character((i-1) %% 500)
  row_idx2 <- as.character((i+9) %% 500)
  # e.g. sim_DE(n = 50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
  sim_temp <- sim_DE(n = 75,
                    size1 = combo[row_idx1, "size"],
                    prob1 = combo[row_idx1, "prob"],
                    size2 = combo[row_idx2, "size"],
                    prob2 = combo[row_idx2, "prob"])
  sim_data_grp1[, i] <- sim_temp$grp1
  sim_data_grp2[, i] <- sim_temp$grp2

  wilcox_tmp <- wilcoxon_wrap(sim_temp)

  wilcoxon_p[i] <- wilcox_tmp$p
  wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

## DP -----

```

```

# [DEG (DP)] Generate 500 paris of data sets for DP,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 8501:9000) {
  row_idx1 <- as.character((i-1) %% 500)
  row_idx2 <- as.character((i+9) %% 500)
  # e.g. sim_DP(50, prop1.1 = 0.3, prop2.1 = 0.8,
  #           size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
  sim_temp <- sim_DP(n = 75,
                    prop1.1 = runif(1, min = 0.20, max = 0.49),
                    prop2.1 = runif(1, min = 0.51, max = 0.80),
                    size1 = combo[row_idx1, "size"],
                    prob1 = combo[row_idx1, "prob"],
                    size2 = combo[row_idx2, "size"],
                    prob2 = combo[row_idx2, "prob"])

  sim_data_grp1[, i] <- sim_temp$grp1
  sim_data_grp2[, i] <- sim_temp$grp2

  wilcox_tmp <- wilcoxon_wrap(sim_temp)

  wilcoxon_p[i] <- wilcox_tmp$p
  wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

## DM -----
# [DEG (DM)] Generate 500 pairs of data sets for DM,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 9001:9500) {
  row_idx1 <- as.character((i-1) %% 500)
  row_idx2 <- as.character((i+9) %% 500)
  # e.g. sim_DM(n = 50, size_cmn = 10, prob_cmn = 0.3,
  #           prop_ext = 0.2, size_ext = 50, prob_ext = 0.1)
  sim_temp <- sim_DM(n = 75,
                    size_cmn = combo[row_idx1, "size"],
                    prob_cmn = combo[row_idx1, "prob"],
                    prop_ext = runif(1, min = 0.20, max = 0.49),
                    size_ext = combo[row_idx2, "size"],
                    prob_ext = combo[row_idx2, "prob"])

  sim_data_grp1[, i] <- sim_temp$grp1
  sim_data_grp2[, i] <- sim_temp$grp2

  wilcox_tmp <- wilcoxon_wrap(sim_temp)

  wilcoxon_p[i] <- wilcox_tmp$p
  wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

## DB -----
# [DEG (DB)] Generate 500 pairs of data sets for DB,
# and perform wilcoxon test at the same time
set.seed(6611)
for (i in 9501:10000) {
  row_idx1 <- as.character((i-1) %% 500)

```

```

row_idx2 <- as.character((i+9) %% 500)
# e.g. sim_DB(50, size1 = 10, prob1 = 0.3, size2 = 50, prob2 = 0.1)
sim_temp <- sim_DB(n = 75,
                   size1 = combo[row_idx1, "size"],
                   prob1 = combo[row_idx1, "prob"],
                   size2 = combo[row_idx2, "size"],
                   prob2 = combo[row_idx2, "prob"])
sim_data_grp1[, i] <- sim_temp$grp1
sim_data_grp2[, i] <- sim_temp$grp2

wilcox_tmp <- wilcoxon_wrap(sim_temp)

wilcoxon_p[i] <- wilcox_tmp$p
wilcoxon_jdg[i] <- wilcox_tmp$DEG
}

# Summary -----
summary(wilcoxon_jdg[1:4000])
summary(wilcoxon_jdg[4001:8000])
summary(wilcoxon_jdg[8001:8500])
summary(wilcoxon_jdg[8501:9000])
summary(wilcoxon_jdg[9001:9500])
summary(wilcoxon_jdg[9501:10000])

# FDR (BH) Correction
wilcoxon_p.adj <- p.adjust(wilcoxon_p, method = "BH")

# save RData -----
save(sim_data_grp1, sim_data_grp2, wilcoxon_p, wilcoxon_jdg, wilcoxon_p.adj,
     file = "midterm_data")

```

Code for Figures & Tables

```

knitr::opts_chunk$set(echo = TRUE)

library(kableExtra) # load for tables
# Figure 1 -----
# Source the draft R file
source("HelperFunctions.R")
library(ggplot2)
library(ggpubr)

# Set seed for reproducibility
set.seed(6611)

# DEG Scenarios & non-DEG Scenarios

param1 <- nbiom_param(mu = 20, variance = 60)

size1 <- param1$size
prob1 <- param1$prob

param2 <- nbiom_param(mu = 80, variance = 400)

```

```

size2 <- param2$size
prob2 <- param2$prob

## Uni-modal -----
sim_uni1 <- sim_unimodal(n = 10000, size = size1, prob = prob1)
Uni1_plot_df <- data.frame(value = sim_uni1)
p_uni <- ggplot(data = Uni1_plot_df, aes(x = value)) +
  geom_histogram(fill = "gray", binwidth = 1) +
  xlab("Expression Counts in Each Cell") +
  ylab("Frequency") +
  ggtitle("Uni-Modal") +
  xlim(c(0, 180))

## Bi-modal -----
sim_bi1 <- sim_bimodal(n = 10000, size1 = size1, prob1 = prob1,
  size2 = size2, prob2 = prob2)
Bi1_plot_df <- data.frame(value = sim_bi1)
p_bi <- ggplot(data = Bi1_plot_df, aes(x = value)) +
  geom_histogram(fill = "gray", binwidth = 1) +
  xlab("Expression Counts in Each Cell") +
  ylab("Frequency") +
  ggtitle("Bi-Modal") +
  xlim(c(0, 180))

## Function: Generate ggplot (for DEG situations)
generate_plot <- function(data_list, title_str) {
  plot_df <- data.frame(value = unlist(data_list),
    group = rep(c("group 1", "group 2"), each = 10000))
  plot_tmp <- ggplot(data = plot_df, aes(x = value, color = group, fill = group)) +
    geom_histogram(position="identity", alpha=0.5, binwidth = 2) +
    scale_color_manual(values=c("#999999", "#E69F00", "#56B4E9")) +
    scale_fill_manual(values=c("#999999", "#E69F00", "#56B4E9")) +
    xlab("Expression Counts in Each Cell") +
    ylab("Frequency") + xlim(c(0, 180)) +
    ggtitle(title_str)
  plot_tmp
}

## EE -----
sim_EE1 <- sim_EE(n = 10000, size = size1, prob = prob1, zero_pct = 0)

EE1_title <- "non-DEG Scenario 1: EE"

p_EE <- generate_plot(sim_EE1, EE1_title)

## EP -----
sim_EP1 <- sim_EP(n = 10000, size1 = size1, prob1 = prob1,
  size2 = size2, prob2 = prob2, zero_pct = 0)
EP1_title <- "non-DEG Scenario 2: EP"

p_EP <- generate_plot(sim_EP1, EP1_title)

```

```

## DE -----
sim_DE1 <- sim_DE(n = 10000, size1 = size1, prob1 = prob1,
                  size2 = size2, prob2 = prob2, zero_pct = 0)

DE1_title <- "DEG Scenario 1: DE"

p_DE <- generate_plot(sim_DE1, DE1_title)

## DP -----
sim_DP1 <- sim_DP(n = 10000, prop1.1 = 0.3, prop2.1 = 0.8,
                  size1 = size1, prob1 = prob1, size2 = size2, prob2 = prob2,
                  zero_pct = 0)
DP1_title <- "DEG Scenario 2: DP"

p_DP <- generate_plot(sim_DP1, DP1_title)

## DM -----
sim_DM1 <- sim_DM(n = 10000, prop_ext = 0.2,
                  size_cmn = size1, prob_cmn = prob1, zero_pct_uni = 0,
                  size_ext = size2, prob_ext = prob2, zero_pct_bi = 0)

DM1_title <- "DEG Scenario 3: DM"
p_DM <- generate_plot(sim_DM1, DM1_title)

## DB -----
sim_DB1 <- sim_DB(n = 10000, size1 = size1, prob1 = prob1,
                  size2 = size2, prob2 = prob2, zero_pct = 0)
DB1_title <- "DEG Scenario 4: DB"
p_DB <- generate_plot(sim_DB1, DB1_title)

# Put all plots together
suppressWarnings(
  ggarrange(p_uni, p_bi, p_EE, p_EP, p_DE, p_DP, p_DM, p_DB,
            labels = LETTERS[1:8],
            ncol = 2, nrow = 4))

# Table 1 -----
# Extract results for null case and scenario with expected 80% power
load("midterm_data")
tab_res <- data.frame(
  Model = c(rep("DEG", 4), rep("non-DEG", 2)),
  Scenario = c("DE", "DP", "DM", "DB", "EE", "EP"),
  Count = c(rep(500, 4), rep(4000, 2)),
  DEG = c(sum(wilcoxon_jdg[8001:8500]), # DE
          sum(wilcoxon_jdg[8501:9000]), # DP
          sum(wilcoxon_jdg[9001:9500]), # DM
          sum(wilcoxon_jdg[9501:10000]), # DB
          sum(wilcoxon_jdg[1:4000]), # EE
          sum(wilcoxon_jdg[4001:8000]) # EP
          ),
  DEG.adj = c(sum(wilcoxon_p.adj[8001:8500] < 0.05), # DE
              sum(wilcoxon_p.adj[8501:9000] < 0.05), # DP
              sum(wilcoxon_p.adj[9001:9500] < 0.05), # DM
              sum(wilcoxon_p.adj[9501:10000] < 0.05), # DB
  )

```

```

        sum(wilcoxon_p.adj[1:4000] < 0.05), # EE
        sum(wilcoxon_p.adj[4001:8000] < 0.05)) # EP
    )

tab_res$NonDEG <- tab_res$Count - tab_res$DEG
tab_res$NonDEG.adj <- tab_res$Count - tab_res$DEG.adj

# tab_res <- tab_res[, c(1:4, 6, 5, 7)]
tab1 <- tab_res[, c(1:3, 5, 7)]

# Create table of results
kbl(tab1,
     caption = 'Summary for Simulation Design and Test Results',
     booktabs=T, align='cc') %>%
kable_paper(full_width = T) %>%
column_spec(1, bold = T) %>%
collapse_rows(columns = 1) # %>%
# add_header_above(c(" " = 3, "Test Results" = 2))
# Figure 2 -----
# TPR for DEG Models
DEG_scenarios <- c("DE", "DP", "DM", "DB")
DEG_scenarios <- factor(DEG_scenarios, levels = DEG_scenarios)

DEG_TPR <- data.frame(scenario = DEG_scenarios,
                     TPR = c(mean(wilcoxon_p.adj[8001:8500] < 0.05), # DE
                           mean(wilcoxon_p.adj[8501:9000] < 0.05), # DP
                           mean(wilcoxon_p.adj[9001:9500] < 0.05), # DM
                           mean(wilcoxon_p.adj[9501:10000] < 0.05) # DB
                           ))

# Bar Plot
DEG_TPR_plot <- ggplot(data = DEG_TPR, aes(scenario, TPR)) + geom_col() +
  ylab("TPR of DEG Models") +
  scale_y_continuous(labels = scales::percent, limits=c(0,1)) +
  geom_text(aes(label=scales::percent(TPR)), vjust=-0.25)

# TNR for non-DEG Models
nonDEG_scenarios <- c("EE", "EP")
nonDEG_scenarios <- factor(nonDEG_scenarios, levels = nonDEG_scenarios)

nonDEG_TNR <- data.frame(scenario = nonDEG_scenarios,
                       TNR = c(mean(wilcoxon_p.adj[1:4000] >= 0.05), # EE
                             mean(wilcoxon_p.adj[4001:8000] >= 0.05) # EP
                             ))

# Bar Plot
nonDEG_TNR_plot <- ggplot(data = nonDEG_TNR, aes(scenario, TNR)) + geom_col() +
  ylab("TNR of non-DEG Models") +
  scale_y_continuous(labels = scales::percent, limits=c(0,1)) +
  geom_text(aes(label=scales::percent(TNR)), vjust=-0.25)

# Put two plots side by side
suppressWarnings(
  ggarrange(DEG_TPR_plot, nonDEG_TNR_plot,
            labels = LETTERS[1:2],
            ncol = 2, nrow = 1))

```